

To utilise the echo action, simply issue the 'echo' keyword underneath the 'watchfor' line as follows:

```
watchfor /su/sudo/
    echo [formatting keywords]
```

Although it is very basic, *echo* offers a variety of formatting methods in which to display alerts. One can change the text colour, underline it, make it bold, strike-through, flash text, as well as combine multiple formatting keywords.

Swatch can alert a user when there is a positive match by issuing a bell sound. The following example illustrates this:

```
watchfor /su/sudo/
    bell [total # of rings]
```

In what is considered as a more efficient way of alerting, one can configure Swatch to send an email whenever there is a match of a specific event. This is a convenient way to alert systems administrators of real-time events without requiring them to be at the console. Let us take a look at an example of this. We will be sending an email to the administrator of a new application that has been installed on the system. Note that the address contains an escape character for the email address \@ — this is required for Perl to format the address correctly. Also, if you would like to send spaces within your subject line, you must place an escape character prior to each space.

```
watchfor /[[i]]nstalled/
    mail addresses=admin_person\@mycompany.com,
    subject=Unauthorized\ Application\ Installation
```

It is important to note here that the *throttle* action helps to prevent denial-of-service attacks via Swatch (e.g., deliberately triggering huge numbers of Swatch events in a short period). In other words, *throttle* gives Swatch the intelligence to ignore repeated occurrences of a given event, potentially preventing minor events from becoming major annoyances.

As part of the remedy, note that Swatch has the ability to execute a secondary script if a specific event has been detected. This functionality could trigger some further action(s) in response to an event. The syntax required uses the *exec* keyword and accepts some bash, Perl, and other useful commands.

The following example illustrates this, where we direct Swatch to execute a remediation Perl script if it detects a port scan against the system.

```
watchfor /[[c]]onnection closed by/
    exec "perl /usr/bin/custom_remediation.pl"
```

Another important scenario where Swatch can be used is to avoid a storm of alert messages. We can configure Swatch to take a specific action only if it detects a certain number of similar events within a certain time frame. Let us consider a

scenario where there are multiple SSH failed login attempts within a specific time period. We can configure Swatch to search for this case and take action — sending a message to the systems administrator as well as outputting a log message to the console. The following commands will handle such a case:

```
watchfor /ssh.*failed/
    echo bold
    mail addresses=admin_person\@mycompany.com,
    subject=Possible\ SSH\ Brute\ Force\ Attempts
    threshold track_by=$1, type=threshold, count=5, seconds=10
```

In the above example, we have set the threshold as '5 failed login attempts within 10 seconds'.

Another powerful feature that Swatch provides is its ability to specify a given time of day that an action will be performed. This is very beneficial to perform certain sets of actions over the weekend or after business hours. To apply a time restriction to an action, we just need to append the keyword 'when=', followed by the time duration after an action. The syntax used to represent the timeframe is indicated in numerical format, whereas each weekday is represented by a number 1-7 (7 = Saturday, 1 = Sunday, etc), and hours are represented as a 24-hour timeframe (1–24). The following example illustrates this concept:

```
watchfor /system full/
    mail addresses=admin_person\@company.com,
    subject=File\ System\ Full,
    when=7-1:1-24
```

In the above example, a user will be alerted by Swatch only over the weekend if a storage drive becomes full.

It is always advisable to follow some best practices when you are using Swatch. If Swatch's actions don't fire very often, it could be that your system isn't getting probed or is misused very much, but it's at least as likely that Swatch isn't casting its net wide enough. In such cases, you may need to periodically scan through your logs manually to see if you're missing anything, and continue to tweak the Swatch configuration file, *.swatchrc*. Also, as another best practice, you should never forget to periodically reconsider the auditing/logging configurations of the daemons that generate log messages in the first place. It is critical to note here that Swatch won't catch events that aren't logged at all. 

By: Swayam Prakasha

The author has a master's degree in computer engineering. He has been working in information technology for several years, concentrating on areas such as operating systems (Linux and UNIX), networking, network security, electronic commerce, Internet services, LDAP and Web servers. He has authored a number of articles for trade publications, and presents his own papers at industry conferences. He can be reached at swayam.prakasha@gmail.com.